

PSP-STD-003

Peach Suite Pro Coding Standards

Version 1.0 Draft | Peach Suite Pro | Internal Standard

Document Number	PSP-STD-003
Status	Draft
Owner	Master Admin
Release Policy	Tuesday Night Releases

1. Purpose

This standard defines coding practices for Peach Suite Pro so the application remains readable, maintainable, secure, and consistent as it grows.

2. Core Coding Principles

- Readable code is preferred over clever code.
- Use consistent patterns across modules.
- Keep business logic in Python, not templates.
- Keep templates focused on display.
- Reuse existing helpers before creating new ones.
- Every route should protect data by spa_id/business ownership.
- Security and data privacy are default requirements, not optional add-ons.

3. Route Standards

- Use descriptive route function names.
- Group related routes together in app.py until modular blueprints are adopted.
- GET routes display pages or confirmation screens.
- POST routes change data.
- Confirmation pages should have separate GET routes when the action is important.
- Always verify current_spa_id ownership before reading or changing records.
- Flash a clear success or warning message after POST actions.

4. Template Standards

- Template filenames should match the route purpose.
- Use existing page containers, form cards, detail rows, and button classes.
- Avoid large inline styles.
- Avoid strong tags unless emphasis is truly needed.
- Do not place database or business logic in templates.
- Use url_for for internal links.

5. Database Standards

- Use parameterized SQL for all database operations.
- Filter business-owned data by `spa_id` where applicable.
- Prefer archive fields such as `is_active` over hard delete when records may be needed later.
- Use `created_at` and `updated_at` where practical.
- Use clear boolean names such as `is_active`, `is_required`, `is_completed`.
- Commit only after successful write operations.
- Close cursors and connections.

6. Naming Standards

- Use `snake_case` for Python functions and variables.
- Use descriptive names over abbreviations.
- Route names should describe the business action.
- Template names should be specific, such as `edit_business_schedule.html`.
- Database columns should be lowercase `snake_case`.

7. Error Handling and User Messages

- If a requested record is not found, flash a warning and redirect safely.
- Do not expose internal errors to users.
- Use clear flash messages: created, updated, archived, restored, deleted.
- Validate required input before writing to the database where needed.

8. Security Standards

- Use `login_required` on protected routes.
- Use `spa_required` where business context is required.
- Never trust `schedule_id`, `client_id`, `appointment_id`, or similar IDs without `spa_id` validation.
- Do not add back-door access.
- Do not expose customer data unnecessarily.
- Do not log sensitive customer data unless specifically required for audit purposes.

9. Git and Development Workflow

- Commit daily work locally.
- Push to backup repository for safe cloud backup when appropriate.
- Push to production only for intentional releases.
- Production releases are planned for Tuesday nights.
- No routine Friday releases.
- Use clear commit messages.

10. Testing Expectations

- Test locally before every commit.
- Test happy path and common failure path.
- Verify navigation links after route changes.

- Verify database writes in pgAdmin or logs when appropriate.
- Run a compile check after Python route changes.
- Smoke test after production deployment.

11. Documentation Expectations

- Update Help Pages when user-facing behavior changes.
- Update Standards when a new repeatable rule is adopted.
- Write Release Notes for production releases.
- Keep the Release Checklist current.

12. Revision History

Version 1.0 - Initial draft. Captures route, template, database, security, Git, testing, and documentation standards.